

Hi

In this segment, we will have a look at the policy evaluation algorithm. Mainly, its implementation and a demo with a gridworld environment.

First of all, we will import some libraries. We are importing the `discrete` module from `gym.envs.toy_text`. This simplifies the implementation of our gridworld environment. We will just specify the basic parameters like state space, action space and transition probabilities, and the discrete module will handle the rest, like the `reset()` and `step()` methods.

The environment setting is like this: the agent starts at a random location in a 4x4 grid. There are two terminal states in this grid as marked by "T". The agent can move in 4 directions: up, down, right and left. The goal is to reach the terminal states as fast as possible. To motivate this behavior, -1 reward is given at each timestep.

In the code, we specify the number of states, actions and transition probabilities for each state-action pair along with the rendering logic.

<should we get into the implementation details of the environment?>

Now, let's recall some details about policy evaluation. To evaluate any given policy is to calculate its value function V^{π} . The value function follows the bellman equation as shown here. We start with any arbitrary value function V_0 and apply this equation to every state to get V_1 . Then we apply this equation to V_1 and get V_2 , and so on.

But how many times should we do this? Theoretically, infinite times. But practically, we see that the changes in the value function become smaller and smaller. So, we stop when such change is smaller than a pre-defined threshold.

As for the algorithm, we start with initial value function which 0 everywhere. Then, for each state, we store its current value and get the new value by applying the equation. Then we compute the change and store it if it is larger than the last change. We do this for every state and repeat. When the change is smaller than a pre-defined threshold, we stop the algorithm. We can see the algorithm in its code form.

Now, as an example, we will evaluate a random policy; for which each action is equally likely in every state.

We get this value function. Now we make sure that it is what we expect.

Next, we plot both the value functions. This is what the calculated and expected value functions look like. Pretty close.

To summarise, we calculated the value function for a random policy, which is called policy evaluation. Later, we will have a look at an algorithm to obtain the optimal policy. Thank you.